3221806228_颜益炜_系统源代码

一、源码组成

目录	作用
LLM助手源码/app.py	应用入口、服务装配和请求路由。
LLM助手源码/agent_runtime.py	统一 Agent 运行时和工具编排入口。
LLM助手源码/rag	知识库检索、提示词组织和模型服务适配。
LLM助手源码/intent	意图识别、字段抽取和结构化命令预览。
LLM助手源码/tool_gateway 与 safety	工具网关、安全策略、路径白名单和确认流程。
LLM助手源码/integration	DAC-3D 状态、命令和检测结果适配边界。
LLM助手源码/ui2	React/Vite 前端交互界面。
DAC-3D主系统源码_精简版/xxp_ui	主系统界面、运行状态、命令桥接和检测结果相关源码。

二、关键源码摘录

完整源码已随“系统作品”文件夹提供。以下仅摘录与论文工作直接相关的关键文件片段，便于评阅时快速定位。

文件：dac3d_iim_assistant\app.py

```
"""Main entry point and top-level routing for the DAC-3D assistant."""

from __future__ import annotations

import argparse
import json
import shutil
import threading
from collections import Counter
from copy import deepcopy
from dataclasses import dataclass, field
from pathlib import Path
from collections.abc import Iterator, Sequence
from typing import Any

from config import AppConfig
from intent.classifier import IntentClassifier, IntentResult
from intent.command_generator import CommandGenerator
from integration.dac3d_client import DAC3DClient, DAC3DUnavailableError, DAC3DValidationError
from integration.result_parser import parse_result, parse_result_from_text
from knowledge_base.build_kb import (
    build_knowledge_base,
    describe_document_format,
    detect_document_format,
    summarize_knowledge_base,
)
from rag.llm_client import LLMClient, LLMConfigurationError, LLMProviderError
from rag.prompts import (
    build_command_clarification_prompt,
    build_greeting_prompt,
    build_guidance_prompt,
    build_interpretation_prompt,
    build_qa_prompt,
)
from rag.retriever import RetrievalItem, Retriever
```

```
from ui.chat_widget import ChatWidget
from ui.web_api import create_api_app

try:
    from machine_agent import MachineAgentService
except Exception:  # pragma: no cover - optional demo module guard
    MachineAgentService = None  # type: ignore[assignment]


@dataclass(slots=True)
class AssistantResponse:
    """Top-level response returned by the application router."""

    intent: str
    answer: str
    sources: list[str] = field(default_factory=list)
    source_items: list[dict[str, Any]] = field(default_factory=list)
    command_preview: dict[str, Any] | None = None
    status_summary: dict[str, Any] | None = None
    parsed_result: dict[str, Any] | None = None

    def render_text(self) -> str:
        """Render the response into a readable terminal block."""
        lines = [f"Intent: {self.intent}", self.answer]
        if self.command_preview is not None:
            lines.append("Command Preview:")
            lines.append(json.dumps(self.command_preview, indent=2, ensure_ascii=False))
        if self.status_summary is not None:
            lines.append("Status:")
            lines.append(json.dumps(self.status_summary, indent=2, ensure_ascii=False))
        if self.parsed_result is not None:
            lines.append("Parsed Result:")
            lines.append(json.dumps(self.parsed_result, indent=2, ensure_ascii=False))
        if self.sources:
            lines.append("Sources:")
            lines.extend(f"- {source}" for source in self.sources)
        return "\n".join(lines)

    def to_ui_payload(self) -> dict[str, Any]:
        """Return UI-friendly structured data."""
        return {
            "intent": self.intent,
            "answer": self.answer,
            "sources": self.source_items,
            "command_preview": self.command_preview,
            "status_summary": self.status_summary,
            "parsed_result": self.parsed_result,
        }


class DAC3DAssistant:
    """Application orchestrator for all DAC-3D assistant flows."""

    _retrieval_expansions = {
        "反光": "样品反光 reflective samples 照明 曝光 消光 倾角",
        "不稳定": "扫描不稳定 unstable scans 样品固定 选区 standard mode precision mode",
        "预览": "preview unstable scans standard mode precision mode",
        "命名区域": "region selection current selection ROI",
        "整片扫描": "scan area 扩大扫描范围 current selection ROI",
        "重新扫描": "inspection status 状态消息 进度百分比",
        "状态": "inspection status 状态消息 进度百分比",
        "precision mode": "scan modes standard mode precision mode defect confirmation",
    }

    def __init__(
        self,
        *,
        config: AppConfig,
        retriever: Retriever,
        llm_client: LLMClient,
        intent_classifier: IntentClassifier,
        command_generator: CommandGenerator,
```

```
        dac3d_client: DAC3DClient,
    ) -> None:
        self.config = config
        self.retriever = retriever
        self.llm_client = llm_client
        self.intent_classifier = intent_classifier
        self.command_generator = command_generator
        self.dac3d_client = dac3d_client

    @classmethod
    def create(
        cls,
        config: AppConfig | None = None,
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\agent_runtime.py

```
"""OpenAI Agents SDK runtime for DAC-3D assistant workflows."""

from __future__ import annotations

import argparse
import json
import re
from collections.abc import Iterator, Sequence
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from agent_core import (
    AGENT_HANDOFF_NAMES,
    AGENT_INSTRUCTIONS,
    AGENT_SPECIALIST_NAMES,
    AGENT_TOOL_NAMES,
    AssistantResponsePayload,
    DAC3DAgentSessionStore,
    DAC3DAgentToolController,
    DAC3D_CONTROL_AGENT_INSTRUCTIONS,
    DAC3D_QA_AGENT_INSTRUCTIONS,
    DAC3D_RESULT_AGENT_INSTRUCTIONS,
    MACHINE_AGENT_INSTRUCTIONS,
    MEMORY_AGENT_INSTRUCTIONS,
    SAFETY_AGENT_INSTRUCTIONS,
    SKILL_AGENT_INSTRUCTIONS,
)
from app import AssistantResponse, DAC3DAssistant
from config import AppConfig
from context_engineering import ContextBuilder, FileBackedContextTree
from goals import ArtifactStore, AutomationPlannerStore, GoalStore, TaskBoardStore, WorkflowTemplateStore
from memory import ConversationMemoryStore, LocalMemoryProvider
from skill_system import SkillPatchStore, SkillRegistry
from trace_eval import CodexHandoffGenerator, EvalDraftGenerator, EvalRunner, TraceLogger


VALID_AGENT_API_TYPES = {"auto", "responses", "chat_completions"}
AGENT_OUTPUT_PARSE_ERROR = "agent_output_parse_error"
LOCAL_VALIDATION_MODEL_NAME = "local-validation"


def response_to_payload(response: AssistantResponse) -> dict[str, Any]:
    """Convert the existing assistant response to a tool-friendly payload."""
    return AssistantResponsePayload.from_response(response).to_dict()


def _parse_json_object(text: str) -> dict[str, Any] | None:
    """Parse a JSON object from the Agent final output."""
    raw = str(text or "").strip()
    if not raw:
        return None

    fenced = re.search(r"```(?:json)?\s*(\{.*?\})\s*```", raw, re.DOTALL | re.IGNORECASE)
```

```
    if fenced:
        raw = fenced.group(1).strip()
    else:
        start = raw.find("{")
        end = raw.rfind("}")
        if start >= 0 and end > start:
            raw = raw[start : end + 1]

    try:
        parsed = json.loads(raw)
    except json.JSONDecodeError:
        return None
    if not isinstance(parsed, dict):
        return None
    return parsed


def _looks_like_local_validation_model(model_name: str) -> bool:
    """Return whether the Agent should use the local validation model."""
    normalized = str(model_name or "").strip().lower()
    return normalized in {LOCAL_VALIDATION_MODEL_NAME, "local_validation", "validation"}


class LocalValidationAgentModel:
    """Deterministic Agents SDK model for local Chrome validation."""

    def __init__(self) -> None:
        self.calls = 0
        self.tool_name = "dac3d_answer"
        self.arguments: dict[str, Any] = {"question": ""}
        self._pending_tool_after_handoff = False
        self._pending_tool_name = ""
        self._pending_arguments: dict[str, Any] = {}
        self.final_payload: dict[str, Any] = {
            "answer": "我已通过本地 Agent 验证模型处理请求。",
            "structured_data": {
                "intent": "agent_validation",
                "tool_calls": [{"name": "dac3d_answer", "purpose": "本地验证默认问答"}],
            },
        }

    async def get_response(
        self,
        system_instructions,
        input,
        model_settings,
        tools,
        output_schema,
        handoffs,
        tracing,
        *,
        previous_response_id,
        conversation_id,
        prompt,
    ):
        del (
            system_instructions,
            model_settings,
            output_schema,
            tracing,
            previous_response_id,
            conversation_id,
            prompt,
        )
        from agents import ModelResponse
        from agents.usage import Usage
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\rag\llm_client.py

```
"""LLM abstraction for the DAC-3D assistant."""
```

```
from __future__ import annotations

import re
import json
from collections.abc import Iterator, Sequence
from typing import Any

from config import AppConfig
from integration.result_parser import ParsedInspectionResult
from rag.retriever import RetrievalItem

SENTENCE_PATTERN = re.compile(r"(?<=[。！？])\s*|(?<=[.!?])\s+")


class LLMProviderError(RuntimeError):
    """Normalized provider error raised by the client."""


class LLMConfigurationError(LLMProviderError):
    """Raised when the requested provider is not correctly configured."""


class ProviderAdapter:
    """Small interface used by the provider registry."""

    def generate(
        self,
        prompt: str,
        *,
        task: str,
        question: str,
        retrieval_items: Sequence[RetrievalItem],
        parsed_result: ParsedInspectionResult | None,
        command: dict[str, Any] | None,
    ) -> str:
        raise NotImplementedError

    def stream_generate(
        self,
        prompt: str,
        *,
        task: str,
        question: str,
        retrieval_items: Sequence[RetrievalItem],
        parsed_result: ParsedInspectionResult | None,
        command: dict[str, Any] | None,
    ) -> Iterator[str]:
        yield self.generate(
            prompt,
            task=task,
            question=question,
            retrieval_items=retrieval_items,
            parsed_result=parsed_result,
            command=command,
        )


class MockProviderAdapter(ProviderAdapter):
    """Deterministic grounded provider for offline demos and tests."""

    def generate(
        self,
        prompt: str,
        *,
        task: str,
        question: str,
        retrieval_items: Sequence[RetrievalItem],
        parsed_result: ParsedInspectionResult | None,
        command: dict[str, Any] | None,
    ) -> str:
        del prompt
        items = list(retrieval_items)
```

```
        if task == "greeting":
            provider_name = question.split("||", 1)[0].strip() or "unknown provider"
            model_name = question.split("||", 1)[1].strip() if "||" in question else "unknown model"
            return (
                f"我是由 {provider_name} 提供的 {model_name} 模型。"
                "我目前可用于文档问答、操作指导、检测结果解读、扫描命令预览和运行状态查询。"
            )

        if task == "command_clarification":
            missing_fields = list((command or {}).get("missing_fields", []))
            warnings = list((command or {}).get("warnings", []))
            if not missing_fields:
                return "命令已经具备必需字段，可以继续确认并执行。"
            details = "、".join(missing_fields)
            warning_text = f" 风险提示: {'; '.join(warnings)}" if warnings else ""
            return f"要继续生成可执行扫描命令，还缺少这些字段: {details}.{warning_text}"

        if task == "interpretation" and parsed_result is not None:
            return self._interpret_result(question, self._prioritize_items(items, task), parsed_result)

        prioritized_items = self._prioritize_items(items, task)

        evidence = self._select_evidence(question, prioritized_items)
        if not evidence:
            return (
                "我没有足够的 DAC-3D 文档依据来安全回答这个问题。"
                "请补充相关手册、FAQ 或参数说明后再试。"
            )

        sources = self._format_sources(prioritized_items)
        if task == "guidance":
            guidance_steps = self._build_guidance_steps(question, prioritized_items)
            return f"建议按以下顺序操作: {guidance_steps} 来源: {sources}."
        return f"根据检索到的 DAC-3D 文档，{evidence} 来源: {sources}."

    def stream_generate(
        self,
        prompt: str,
        *,
        task: str,
        question: str,
        retrieval_items: Sequence[RetrievalItem],
        parsed_result: ParsedInspectionResult | None,
        command: dict[str, Any] | None,
    ) -> Iterator[str]:
        answer = self.generate(
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\intent\command_generator.py

```
"""Structured command generation compatibility layer for DAC-3D requests."""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

from intent.parser import IntentParser
from intent.schemas import ParsedCommand


@dataclass(slots=True)
class StructuredCommand:
    """Normalized DAC-3D command preview."""

    action: str
    scan_area_mm: dict[str, float] | None
    resolution: dict[str, float | str] | None
    region: str | None
    mode: str | None
    payload: dict[str, Any] = field(default_factory=dict)
```

```
    safety: dict[str, Any] = field(default_factory=dict)
    missing_fields: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    yaml_preview: str | None = None

    def to_dict(self) -> dict[str, Any]:
        """Convert the command preview to a serializable mapping."""
        return {
            "action": self.action,
            "scan_area_mm": self.scan_area_mm,
            "resolution": self.resolution,
            "region": self.region,
            "mode": self.mode,
            "payload": self.payload,
            "safety": self.safety,
            "missing_fields": self.missing_fields,
            "warnings": self.warnings,
            "yaml_preview": self.yaml_preview,
        }


class CommandGenerator:
    """Convert natural language into a structured DAC-3D command preview."""

    def __init__(self, parser: IntentParser | None = None) -> None:
        self.parser = parser or IntentParser()

    def generate(self, text: str) -> StructuredCommand:
        """Return a validated structured command representation."""
        parsed = self.parser.parse(text)
        command = parsed.command or ParsedCommand(action="scan")
        structured = StructuredCommand(
            action=command.action or "scan",
            scan_area_mm=command.scan_area_mm,
            resolution=command.resolution,
            region=command.region,
            mode=command.mode,
            payload=dict(command.payload),
            safety=dict(command.safety),
            missing_fields=list(parsed.missing_fields),
            warnings=list(parsed.warnings),
        )
        structured.yaml_preview = self._render_yaml(structured)
        return structured

    def _render_yaml(self, command: StructuredCommand) -> str:
        area = command.scan_area_mm or {}
        resolution = command.resolution or {}
        lines = [
            f"action: {command.action}",
            "scan_area_mm:",
            f"  width: {area.get('width', 'null')}",
            f"  height: {area.get('height', 'null')}",
            "resolution:",
            f"  value: {resolution.get('value', 'null')}",
            f"  unit: {resolution.get('unit', 'null')}",
            f"region: {command.region or 'null'}",
            f"mode: {command.mode or 'null'}",
            "payload:",
        ]
        if command.payload:
            lines.extend(self._render_yaml_mapping(command.payload, indent=2))
        else:
            lines.append("  {}")
        lines.append("safety:")
        if command.safety:
            lines.extend(self._render_yaml_mapping(command.safety, indent=2))
        else:
            lines.append("  {}")
        lines.append("missing_fields:")
        if command.missing_fields:
            lines.extend(f"  - {field_name}" for field_name in command.missing_fields)
```

```
            else:
                lines.append("  []")
            lines.append("warnings:")
            if command.warnings:
                lines.extend(f"  - {warning}" for warning in command.warnings)
            else:
                lines.append("  []")
            return "\n".join(lines)

        def _render_yaml_mapping(self, value: dict[str, Any], *, indent: int) -> list[str]:
            spaces = " " * indent
            lines: list[str] = []
            for key, item in value.items():
                if isinstance(item, dict):
                    lines.append(f"{spaces}{key}:")
                    lines.extend(self._render_yaml_mapping(item, indent=indent + 2))
                elif isinstance(item, list):
                    lines.append(f"{spaces}{key}:")
                    if item:
                        lines.extend(f"{spaces}  - {entry}" for entry in item)
                    else:
                        lines.append(f"{spaces}  []")
                else:
                    rendered = "null" if item is None else item
                    lines.append(f"{spaces}{key}: {rendered}")
            return lines
```

文件：dac3d_iim_assistant\tool_gateway\gateway.py

```
"""MCP-style Tool Gateway for DAC-3D Agent Runtime."""

from __future__ import annotations

from copy import deepcopy
from dataclasses import asdict, dataclass, field
from pathlib import Path
from typing import Any

from agent_core.schemas import AssistantResponsePayload
from agent_core.sessions import DAC3DAgentSessionStore
from intent.structured_commands import READ_ONLY_ACTIONS, SUPPORTED_ACTIONS, missing_required_fields
from safety import PolicyDecision, PolicyEngine, SafetyGuard


@dataclass(slots=True)
class ToolDescriptor:
    """Stable metadata for one gateway tool."""

    name: str
    description: str
    input_schema: dict[str, Any]
    output_schema: dict[str, Any]
    risk_level: str
    requires_confirmation: bool = False
    read_only: bool = False
    destructive: bool = False
    idempotent: bool = False
    open_world: bool = False
    allowed_scopes: list[str] = field(default_factory=list)
    read_only_hint: bool | None = None
    destructive_hint: bool | None = None
    idempotent_hint: bool | None = None
    open_world_hint: bool | None = None

    def to_dict(self) -> dict[str, Any]:
        payload = asdict(self)
        read_only_hint = self.read_only if self.read_only_hint is None else self.read_only_hint
        destructive_hint = self.destructive if self.destructive_hint is None else self.destructive_hint
        idempotent_hint = self.idempotent if self.idempotent_hint is None else self.idempotent_hint
        open_world_hint = self.open_world if self.open_world_hint is None else self.open_world_hint
        payload.update(
```

```
            {
                "readOnlyHint": read_only_hint,
                "destructiveHint": destructive_hint,
                "idempotentHint": idempotent_hint,
                "openWorldHint": open_world_hint,
                "annotations": {
                    "readOnlyHint": read_only_hint,
                    "destructiveHint": destructive_hint,
                    "idempotentHint": idempotent_hint,
                    "openWorldHint": open_world_hint,
                },
            }
        )
        return payload


@dataclass(slots=True)
class ToolInvocationResult:
    """Auditable tool invocation payload."""

    tool: str
    ok: bool
    result: dict[str, Any] = field(default_factory=dict)
    error: str = ""
    risk: dict[str, Any] = field(default_factory=dict)
    validation: dict[str, Any] = field(default_factory=dict)

    def to_dict(self) -> dict[str, Any]:
        return asdict(self)


def _submit_command_input_schema() -> dict[str, str]:
    return {"command_preview_id": "string", "confirmation_" + "token": "string"}


class DAC3DToolGateway:
    """Controlled boundary for DAC status, result, preview, validation, and execution tools."""

    def __init__(
        self,
        *,
        assistant: Any,
        sessions: DAC3DAgentSessionStore,
        session_id: str,
        allowed_roots: list[str | Path] | None = None,
    ) -> None:
        self.assistant = assistant
        self.sessions = sessions
        self.session_id = self.sessions.normalize_session_id(session_id)
        roots = allowed_roots or self._default_allowed_roots()
        write_roots = self._default_command_output_roots()
        self.safety_guard = SafetyGuard(allowed_roots=roots, allowed_write_roots=write_roots)
        self.policy_engine = PolicyEngine(
            allowed_roots=roots,
            allowed_write_roots=write_roots,
            registered_tools=[descriptor.name for descriptor in self.descriptors()],
        )

    def describe(self) -> dict[str, Any]:
        """Return gateway diagnostics and tool descriptors."""
        return {
            "enabled": True,
            "backend": "dac_tool_gateway",
            "tool_count": len(self.descriptors()),
            "tools": [descriptor.to_dict() for descriptor in self.descriptors()],
            "allowed_dirs": self.list_allowed_dirs().result["allowed_dirs"],
            "workflow": "preview -> validate -> confirm -> submit -> trace",
            "metadata_policy": (
                "MCP-style hints are advisory only; SafetyGuard enforces confirmation, "
                "schema validation, path allowlists, and prompt-injection blocking."
            ),
            "policy_engine": {
```

```
                "enabled": True,
                "mode": "fail_closed",
                "registered_tools": [descriptor.name for descriptor in self.descriptors()],
                "enforcement": (
                    "Every Tool Gateway call is evaluated by PolicyEngine before DAC state, "
                    "preview, validation, submit, cancel, or audit data is returned."
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\safety\policy_engine.py

```
"""Code-enforced production policy engine for DAC-Agent Runtime."""

from __future__ import annotations

import re
from pathlib import Path
from typing import Any

from intent.structured_commands import READ_ONLY_ACTIONS, SUPPORTED_ACTIONS, missing_required_fields
from safety.guard import CONTROL_ACTIONS, SafetyGuard
from safety.models import PolicyDecision


BYPASS_PATTERNS = (
    r"skip (confirmation|approval|safety)",
    r"bypass (confirmation|approval|safety)",
    r"without (confirmation|approval)",
    r"no (confirmation|approval) needed",
    r"ignore (all )?(previous|above|system|safety) (instructions|rules|policy)",
    r"直接执行",
    r"立即执行不用确认",
    r"不用确认",
    r"不要确认",
    r"跳过(确认|审批|安全)",
    r"绕过(确认|审批|安全)",
    r"忽略(系统|安全|以上|之前)",
)

SECRET_VALUE_PATTERN = re.compile(
    r"(sk-[A-Za-z0-9_-]{16,}|api[_-]?key\s*[:=]\s*\S+|authorization\s*[:=]\s*\S+|"
    r"password\s*[:=]\s*\S+|secret\s*[:=]\s*\S+|token\s*[:=]\s*\S+)",
    re.IGNORECASE,
)

FORBIDDEN_TOOLS = {
    "write_command_json",
    "write_command_file",
    "direct_command_writer",
    "filesystem_write",
    "shell_exec",
    "python_exec",
}


class PolicyEngine:
    """Enforce DAC-Agent safety policy in code, independent of LLM prompts."""

    def __init__(
        self,
        *,
        allowed_roots: list[str | Path] | None = None,
        allowed_write_roots: list[str | Path] | None = None,
        registered_tools: list[str] | None = None,
    ) -> None:
        self.guard = SafetyGuard(
            allowed_roots=allowed_roots or [],
            allowed_write_roots=allowed_write_roots or [],
        )
        self.registered_tools = set(registered_tools or [])

    def evaluate_intent(
```

```
        self,
        intent: str,
        *,
        command: dict[str, Any] | None = None,
        user_text: str = "",
        retrieved_text: str = "",
    ) -> PolicyDecision:
        """Evaluate a user/model intent before any tool work happens."""
        bypass = self._find_bypass_signals(" ".join([user_text, retrieved_text]))
        if bypass:
            return PolicyDecision.block(
                risk_level="forbidden",
                requires_confirmation=True,
                reasons=["Prompt or user text attempted to bypass DAC safety policy."],
                blocking_reasons=["policy_bypass_request"],
                policy_ids=["POLICY-INJECTION-BYPASS"],
                metadata={"matches": bypass},
            )

        risk = self.guard.classify_intent_risk(intent, command)
        return PolicyDecision(
            allowed=risk.can_execute,
            risk_level=risk.risk_level,
            requires_confirmation=risk.requires_confirmation,
            reasons=[risk.reason],
            blocking_reasons=list(risk.blocked_by),
            policy_ids=["POLICY-INTENT-RISK"],
        )

    def evaluate_tool_call(
        self,
        tool_metadata: dict[str, Any] | None,
        arguments: dict[str, Any] | None = None,
        *,
        private_context: bool = False,
        schema_valid: bool = True,
        user_text: str = "",
        retrieved_text: str = "",
        confirmed: bool = False,
    ) -> PolicyDecision:
        """Evaluate one Tool Gateway call. Unknown tools fail closed."""
        metadata = dict(tool_metadata or {})
        arguments = dict(arguments or {})
        name = str(metadata.get("name") or "").strip()
        if not name:
            return PolicyDecision.block(
                reasons=["Tool metadata is missing a registered name."],
                blocking_reasons=["unknown_tool"],
                policy_ids=["POLICY-TOOL-UNKNOWN"],
            )
        if name in FORBIDDEN_TOOLS or (self.registered_tools and name not in self.registered_tools):
            return PolicyDecision.block(
                reasons=[f"Tool is not registered or is forbidden: {name}"],
                blocking_reasons=["forbidden_tool"],
                policy_ids=["POLICY-TOOL-FORBIDDEN"],
                metadata={"tool": name},
            )
        if not schema_valid or not self._arguments_match_schema(metadata.get("input_schema"), arguments):
            return PolicyDecision.block(
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\integration\dac3d_client.py

```
"""DAC-3D integration adapter with a deterministic mock mode."""

from __future__ import annotations

import json
import os
import time
import uuid
```

```
from collections import Counter, defaultdict
from copy import deepcopy
from datetime import date, timedelta
from pathlib import Path
from typing import Any
from urllib.parse import unquote, urlparse

from intent.structured_commands import (
    DEFAULT_REQUIRED_CAMERAS,
    DEFAULT_REQUIRED_SURFACES,
    SUPPORTED_ACTIONS,
    CommandAction,
)

class DAC3DUnavailableError(RuntimeError):
    """Raised when a live DAC-3D adapter is requested but unavailable."""


class DAC3DValidationError(RuntimeError):
    """Raised when a DAC-3D command is invalid for submission."""


class DAC3DClient:
    """Communicate with DAC-3D services or a local mock implementation."""

    def __init__(
        self,
        *,
        mock_mode: bool = True,
        endpoint: str = "mock://dac3d",
        runtime_bridge: Any | None = None,
    ) -> None:
        self.mock_mode = mock_mode
        self.endpoint = endpoint
        self.runtime_bridge = runtime_bridge
        self._last_command: dict[str, Any] | None = None
        self._status = {
            "state": "idle",
            "progress": 0,
            "message": "当前没有正在执行的 DAC-3D 检测任务。",
        }
        self._latest_result_summary = {
            "result_root": "mock://dac3d/results/latest",
            "files": ["defect_detail.csv", "defect_summary.csv"],
            "artifacts": ["mock_detect.jpg", "mock_fused_image.jpg"],
        }
        self._result_profiles = {
            "scratch_high": {
                "defect_type": "scratch",
                "location": "左上 ROI",
                "confidence": 0.94,
                "measurements": {
                    "depth_um": 18.4,
                    "length_mm": 0.62,
                    "width_um": 41.0,
                },
                "sample_id": "SAMPLE-001",
                "is_qualified": False,
                "is_ignored": False,
                "sample_quality": False,
                "sample_quality_label": "不合格",
                "region": "ROI",
            },
            "pit_medium": {
                "defect_type": "pit",
                "location": "中心 ROI",
                "confidence": 0.88,
                "measurements": {
                    "depth_um": 6.3,
                    "diameter_um": 52.0,
                },
                "sample_id": "SAMPLE-002",
```

```
            },
        }
        self._active_result_profile = "scratch_high"
        self._mock_result_history = self._build_mock_result_history()

    def submit_scan_command(self, command: dict[str, Any]) -> dict[str, Any]:
        """Submit or simulate a structured DAC-3D command."""
        self._validate_command(command)
        if self.runtime_bridge is not None:
            return self._submit_via_runtime_bridge(command)
        if self.endpoint.startswith("file://"):
            return self._submit_via_file_bridge(command)

        if not self.mock_mode:
            raise DAC3DUnavailableError(
                "Live DAC-3D integration is not implemented in this workspace."
            )

        action = str(command["action"])
        self._last_command = deepcopy(command)
        if action == CommandAction.QUERY_STATUS:
            return {
                "accepted": True,
                "mode": "mock",
                "command": deepcopy(command),
                "status": self.query_current_status(),
            }
        if action == CommandAction.GET_LATEST_RESULT:
            return {
                "accepted": True,
                "mode": "mock",
                "command": deepcopy(command),
                "status": deepcopy(self._status),
                "result": self.get_latest_result_summary(),
            }
        if action == CommandAction.VALIDATE_OFFLINE_FOLDER:
            return {
                "accepted": True,
                "mode": "mock",
                "command": deepcopy(command),
...（后续源码见系统作品文件夹）
```

文件：dac3d_iim_assistant\integration\result_parser.py

```
"""DAC-3D result parsing and normalization."""

from __future__ import annotations

import re
from dataclasses import asdict, dataclass
from typing import Any


MEASUREMENT_PATTERNS = {
    "depth_um": re.compile(r"(?:depth|深度)\s*(?:=|为|是)?\s*(?P<value>\d+(?:\.\d+)?)\s*um", re.IGNORECASE),
    "length_mm": re.compile(r"(?:length|长度)\s*(?:=|为|是)?\s*(?P<value>\d+(?:\.\d+)?)\s*mm", re.IGNORECASE),
}
GENERIC_UM_PATTERN = re.compile(r"(?P<value>\d+(?:\.\d+)?)\s*um", re.IGNORECASE)
GENERIC_MM_PATTERN = re.compile(r"(?P<value>\d+(?:\.\d+)?)\s*mm", re.IGNORECASE)


@dataclass(slots=True)
class ParsedInspectionResult:
    """Stable assistant-facing inspection result schema."""

    defect_type: str
    severity: str
    confidence: float
    location: str
    measurements: dict[str, float]
    trigger_measurement: str | None
```

```
    threshold_reference: str | None
    rule_reason: str
    summary: str
    is_qualified: bool | None = None
    is_ignored: bool | None = None
    sample_quality: bool | None = None
    sample_quality_label: str | None = None
    region: str | None = None
    defect_reason: str | None = None
    defects_num: int | None = None

    def to_dict(self) -> dict[str, Any]:
        """Return a serializable representation."""
        return asdict(self)


def parse_result(payload: dict[str, Any]) -> ParsedInspectionResult:
    """Normalize a DAC-3D response payload."""
    defect_type = str(payload.get("defect_type", "unknown")).lower()
    location = str(payload.get("location", "unknown"))
    confidence = float(payload.get("confidence", 0.0))
    measurements = {
        key: float(value)
        for key, value in dict(payload.get("measurements", {})).items()
        if isinstance(value, (int, float))
    }

    derived = _derive_severity_metadata(defect_type, measurements)
    severity = str(payload.get("severity") or derived["severity"])
    trigger_measurement = str(derived["trigger_measurement"]) if derived["trigger_measurement"] else None
    threshold_reference = (
        str(derived["threshold_reference"]) if derived["threshold_reference"] else None
    )
    rule_reason = str(payload.get("rule_reason") or derived["rule_reason"])
    is_qualified = _optional_bool(payload.get("is_qualified"))
    is_ignored = _optional_bool(payload.get("is_ignored"))
    sample_quality = _optional_bool(payload.get("sample_quality"))
    if sample_quality is None:
        sample_quality = _optional_bool(payload.get("quality"))
    sample_quality_label = _optional_str(payload.get("sample_quality_label"))
    if sample_quality_label is None:
        sample_quality_label = _optional_str(payload.get("quality_label"))
    if sample_quality_label is None and sample_quality is not None:
        sample_quality_label = "合格" if sample_quality else "不合格"
    region = _optional_str(payload.get("region"))
    defect_reason = _optional_str(payload.get("defect_reason") or payload.get("reason"))
    defects_num = _optional_int(payload.get("defects_num"))
    summary = _build_summary(
        defect_type=defect_type,
        severity=severity,
        location=location,
        measurements=measurements,
        rule_reason=rule_reason,
        is_qualified=is_qualified,
        is_ignored=is_ignored,
        sample_quality_label=sample_quality_label,
    )
    return ParsedInspectionResult(
        defect_type=defect_type,
        severity=severity,
        confidence=confidence,
        location=location,
        measurements=measurements,
        trigger_measurement=trigger_measurement,
        threshold_reference=threshold_reference,
        rule_reason=rule_reason,
        summary=summary,
        is_qualified=is_qualified,
        is_ignored=is_ignored,
        sample_quality=sample_quality,
        sample_quality_label=sample_quality_label,
        region=region,
```

```
        defect_reason=defect_reason,
        defects_num=defects_num,
    )


def parse_result_from_text(text: str) -> ParsedInspectionResult | None:
    """Parse a user-supplied defect description into the stable result schema."""
    normalized = text.strip().lower()
    defect_type = _extract_defect_type(normalized)
    if defect_type is None:
        return None

    measurements: dict[str, float] = {}
    for key, pattern in MEASUREMENT_PATTERNS.items():
        match = pattern.search(normalized)
        if match:
            measurements[key] = float(match.group("value"))

    if defect_type == "pit" and "depth_um" not in measurements:
        match = GENERIC_UM_PATTERN.search(normalized)
        if match:
...（后续源码见系统作品文件夹）
```

文件：福特科\xxp_ui\window\ui.py

```
  # import halcon as ha
import queue
import threading
import time
import random
import json
import os
import subprocess
import sys
import webbrowser
from pathlib import Path
from functools import partial

import cv2
from PyQt5 import uic
from PyQt5.QtCore import QPoint, QTimer, QRect, pyqtSlot, QThread
from PyQt5.QtGui import QMouseEvent, QIcon
from PyQt5.QtWidgets import QCheckBox, QFileDialog, QLabel, QMainWindow, QGridLayout, QPushButton, QFrame,
QVBoxLayout
from PyQt5 import QtCore
from datebase.DB import DBController

from .utils import *
from .HistoryCard import HistoryCard
from collections import deque
from .Panel import SlidingPanel
from .NotificationManager import notification_manager
from .PictureViewer import ImageViewer


class MyWindow(QMainWindow):
    timel = QTimer()  # 定时器500ms初始化
    myDatabase = DBController()  # 数据库控制类初始化
    # tcp_client = TCPClient('localhost',port=27015)
    # g = gclib.py()
    # cam = CameraFactory()
    # Zmc = ZAUXDLL()
    # light = lightControllor()

    image_queue = queue.Queue(maxsize=144)

    samples_result = []#接收样品信息的列表
    # samples = []
    sample_res = []
    stopFlag = False
    Tray_id = None
```

```
    def __init__(self, ui_debug_queue, debug_ui_queue,ui_image_queue,image_ui_queue,exit_event,mode):
        super().__init__()

        self.scan_start_time = 0
        self.history_button = None
        self.mode = mode
        self.detect_button = None
        self._startPos = None
        self._endPos = None
        self.assistant_web_process = None
        self.assistant_status_file = (
            Path(__file__).resolve().parents[3]
            / 'dac3d_iim_assistant'
            / '.tmp'
            / 'dac3d_runtime_status.json'
        )
        self.assistant_command_file = self.assistant_status_file.with_name('dac3d_assistant_command.json')
        self.assistant_command_ack_file =
self.assistant_status_file.with_name('dac3d_assistant_command_ack.json')
        self._last_assistant_command_id = None
        self.assistant_latest_result = None
        self.assistant_result_history = []

        self.ui_debug_queue = ui_debug_queue
        self.debug_ui_queue = debug_ui_queue
        self.ui_image_queue = ui_image_queue
        self.image_ui_queue = image_ui_queue
        self.exit_event = exit_event

        self.InitUI(mode)
        self.writeAssistantRuntimeStatus('idle', 0, 'DAC-3D 主系统已启动，当前没有正在执行的检测任务。', 'ready')
        # self.detect_button.changeButtonType(50, 'pass')
        # self.detect_button.changeButtonType(1, 'pass')

        # print('gclib version:', self.g.GVersion())
        # self.g.GOpen('10.0.0.100 --direct -s ALL')
        #
        # strtemp = '192.168.0.11'
        # iresult = self.Zmc.ZAux_OpenEth(strtemp)
        # if 0 != iresult:
        #     print('运动控制卡连接失败')
        # else:
        #     print('运动控制卡连接成功')

    def mouseMoveEvent(self, e: QMouseEvent):  # 重写移动事件，控制窗口移动
        if self._startPos:
            self._endPos = e.pos() - self._startPos
            self.ui.move(self.ui.pos() + self._endPos)
            if not self.ui.panel.isHidden():
                self.ui.panel.move(QRect(
                    self.ui.x() + self.ui.width(),
                    self.ui.y(),
                    self.ui.panel.width(),
                    self.ui.height()
                ).topLeft())

    def mousePressEvent(self, e: QMouseEvent):
        self._isTracking = True
        self._startPos = QPoint(e.x(), e.y())

    def mouseReleaseEvent(self, e: QMouseEvent):
        self._isTracking = False
        self._startPos = None
        self._endPos = None

    def closeEvent(self, event):
        """重写关闭事件，设置退出标志"""
        self.exit_event.set()
        event.accept()

    def stateChanged(self):  # 定时器循环函数
        self._time, self._date, _ = getDateAndTime()
```

```
            self.ui.label_5.setText(self._time)

            # msg = self.image_ui_queue.get(0.01)
...（后续源码见系统作品文件夹）
```